COMP719 – OPTIMIZATION AND MODELLING SEMESTER PROJECT

PROJECT TITLE: THE CAMPUS LAB SPACE OPTIMIZER

PHASE: PHASE 1 – PROPOSAL AND INITIAL MODEL

1. GROUP INFORMATION

Group Members and Contribution Estimates:

Member Name	Student ID	Role	% Contribution
Jeryn Naidoo	223009245	Software engineering and Presentation Slides	20
Kirone Gopaul	223014456	Software Engineering	20
Shaista Naicker	222106172	Documentation and Presentation slides	20
Josh Goodwin	222092144	Documentation	20
Tashmika Pillay	223097300	Software Testing and Documentation	20

2. PROBLEM UNDERSTANDING

2.1 Project Summary

The Campus Lab Space Optimizer addresses the challenge of allocating university modules to computer lab in a way that maximizes resource efficiency. Each module must be assigned to a computer lab with sufficient seating capacity and the respective software installed. Since labs differ in both size and software configuration, straightforward or manual allocations often lead to wasted seating capacity or incompatible assignments.

The objective of the project is to optimize lab usage by assigning every module to a single compatible lab while minimizing the total number of unused seats across all allocations. A good solution must guarantee feasibility by ensuring that no module is placed in a lab that is too small or lacks the required software, while also improving efficiency by preventing smaller classes from occupying large labs unnecessarily. The challenge lies in striking a balance between feasibility and underutilization using a structured optimization approach.

2.2 Key Challenges

One of the main challenges is ensuring all constraints are enforced simultaneously. Each module-lab assignment must satisfy both seating capacity and software installation requirements. If either of these constraints is overlooked or implemented incorrectly, the solution becomes infeasible, even if it appears reasonable to the human eye.

A second challenge is reducing the total number of unused seats without preventing future assignments. Greedy decisions may appear efficient at first, but can lead to problems later, particularly when larger or more constrained modules are left with no suitable labs. Balancing the minimisation of seat wastage with the need to preserve feasible options for remaining modules requires careful ordering and well-designed algorithmic decision-making.

Additionally, a challenge lies in handling modules with limited placement options. Some modules may only be compatible with a few labs due to a high volume of students or specific software requirements. If these modules are not prioritised correctly, the algorithm may fail even if a valid solution exists.

Finally, designing a greedy rule that performs consistently well is a challenge. Greedy algorithms don't reevaluate past decisions, so the optimality of the final solution depends on the initial strategy and tie-breaking rules. This makes it challenging to choose a heuristic that works well across different datasets, which is why further refinement is considered in later phases of the project.

3. DATA UNDERSTANDING

3.1 Description of Labs

Each lab is described by four attributes: a lab ID, a descriptive name, seating capacity, and a set of software packages installed. The capacity determines whether a lab can physically accommodate a module's students. The software determines if the lab is compatible with the module's requirements. The descriptive name serves as unique identifier, whilst the lab ID is used to make reference to a specific lab unambiguously.

3.2 Description of Modules

Each module is described by four attributes, similarly to the labs' descriptions: a module code that serves as unique identifier, a descriptive name, the number of enrolled students, and a single required software package. The enrolled students

determine whether a module can be allocated to a lab, while the software requirement determines whether the lab can support the module.

The attributes, for both modules and labs, are essential for enforcing feasibility constraints and computing unused seats.

4. MATHEMATICAL MODEL

4.1 Decision Variables

$$x_{ij} = \begin{cases} 1 & \text{if module } i \text{ is assigned to lab } j \\ 0 & \text{otherwise} \end{cases}$$

Where:

- $i \in \{M1, M2, \ldots, M10\}$ — the set of modules
- $j \in \{L1, L2, L3, L4\}$ — the set of labs

The decision variable x_{ij} indicates whether a specific module is assigned to a specific lab. If module i is placed in lab, the variable takes the value 1, otherwise takes the value 0. These variables collectively represent a complete assignment of modules to labs.

4.2 Constraints

Constraint 1: Unique Assignment

$$\sum_{j} x_{ij} = 1 \quad \forall i$$

> Every module must be scheduled in one lab for the semester — no module can be unassigned or assigned to two labs.

This constraint ensures that each module is assigned to exactly one lab. A module cannot be left without a lab, and it cannot be assigned to more than one lab. This represents the real-world requirement that ever module must be scheduled in a single computer lab.

Constraint 2: Capacity Constraint

$$\text{If } \text{Students}_i > \text{Capacity}_j, \text{ then } x_{ij} = 0$$

Or equivalently:

$$x_{ij} = 0 \quad \text{if } \text{Students}_i > \text{Capacity}_j$$

> No student should be left without a seat.

This constraint prevents a module from being assigned to a lab with insufficient seats. If

the number of students enrolled in a module exceeds the seating capacity of a lab, a module cannot be assigned to that lab. This ensures all students are accommodated.

Constraint 3: Software Compatibility

$$x_{ij} = 0 \quad \text{if SoftwareNeeded}_i \notin \text{SoftwareInstalled}_j$$

> The lab must have the specific software the module requires.

This constraint ensures that a module can only be assigned to a lab that has the required software installed. If the lab is not software compatible with that module, the assignment is not permitted. This guarantees that teaching requirements can be met.

Constraint 4: Binary Nature of Variables

$$x_{ij} \in \{0, 1\} \quad \forall i, j$$

This constraint specifies that decision variables are binary representing that a module is either assigned to a lab (1) or not (0).

4.3 Objective Function

For each assigned module–lab pair, unused seats are:

$$\text{UnusedSeats}_{ij} = \text{Capacity}_j - \text{Students}_i$$

The total unused seats across all assignments:

$$Z = \sum_i \sum_j (\text{Capacity}_j - \text{Students}_i) \cdot x_{ij}$$

Minimize:

$$\min Z = \min \sum_i \sum_j (\text{Capacity}_j - \text{Students}_i) \cdot x_{ij}$$

Subject to all constraints above.

Unused seats represent the number of empty seats, calculated as the difference between the seating capacity of the lab and the students enrolled in that module. This measures how efficiently the lab's capacity is utilised. The objective of the model is to minimise the total number of unused seats across all labs. We multiply the unused seats for each possible assignment by the decision variable x_{ij} so that only the selected assignments contribute to the total. Minimizing this sum results in resource efficiency while respecting all constraints.

5. PROPOSED SOLUTION APPROACH - Shaista

5.1 Feasibility Checking Strategy

For capacity, we compare the students enrolled in a module against each lab's capacity and rule out any lab where the students wouldn't fit. For software, we check whether the lab's installed software can support the module's requirements and rule out the incompatible labs. The remaining labs after both checks form the feasibility region, which is labs we'd consider assigning that module to.

5.2 Proposed Greedy Idea (Initial Thinking)

Our greedy approach works by sorting modules in descending order based on the student count. For each module, we then check software compatibility to rule out any labs that can't meet the requirement. From the remaining valid labs, we choose the one with the smallest capacity that still fits the class. This is known as a Best-Fit Decreasing strategy, similar to classic bin-packing heuristics.

The logic behind this is simple. Larger modules have fewer labs that can accommodate them, so leaving them until later poses the risk of all suitable labs already being taken by smaller modules that didn't need as much space. By sorting largest first, we ensure the more constrained modules are given priority before options run out.

When it comes to selecting a lab, we always go for the smallest one suitable. That way we're not allocating a small class to a large lab and wasting seats unnecessarily, which is directly what the objective function is asking us to minimize. It's a greedy approach because at each step, we're making the best available decision without going back to reconsider earlier ones, giving us a solid starting point to improve on later.

6. IMPLEMENTATION PLAN

6.1 Programming Tools

The project is implemented using Python version – 3.13, and the only external library used at this stage is Pandas, which enables a structured, tabular representation of the data.

6.2 Data Structures

We make use of the Pandas library to convert the Python raw dictionaries, where each key corresponds to an attribute, into Pandas DataFrame objects. This allows us to easily manipulate data, as well as simplify the process of checking the data against the required constraints. We make use of data frames to store both the Lab data and the Module data. To track compatibility, we use a function that checks both the lab's capacity and installed software for each module. The results are stored in the module DataFrame, where each entry is a list of compatible lab IDs. Assignments of modules to labs will be stored in a separate dictionary or new column in the module DataFrame, mapping a module to its allocated lab.

```
df_labs = pd.DataFrame(labs_data)
df_modules = pd.DataFrame(modules_data)
```

	Lab ID	Name	Capacity	Software
0	L1	Advanced Lab A	100	[Python, Java, MATLAB]
1	L2	Coding Lab B	60	[Python, Java]
2	L3	Data Lab C	40	[Python, MATLAB]
3	L4	Legacy Lab D	30	[Java, MATLAB]

	Module Code	Students	Software Needed
0	M1	85	Python
1	M2	45	MATLAB
2	M3	25	Java
3	M4	55	Python
4	M5	35	MATLAB
5	M6	15	Java
6	M7	90	Java
7	M8	30	Python
8	M9	20	MATLAB
9	M10	50	Python

	Module Code	Students	Software Needed	Compatible Labs
0	M1	85	Python	[L1]
1	M2	45	MATLAB	[L1]
2	M3	25	Java	[L1, L2, L4]
3	M4	55	Python	[L1, L2]
4	M5	35	MATLAB	[L1, L3]
5	M6	15	Java	[L1, L2, L4]
6	M7	90	Java	[L1]
7	M8	30	Python	[L1, L2, L3]
8	M9	20	MATLAB	[L1, L3, L4]
9	M10	50	Python	[L1, L2]

```
def display_compatability_info():
    compatible_labs = []

    for m_idx in range(len(df_modules)):

        current_compatible_labs = []

        for l_idx in range(len(df_labs)):

            if(check_fit(m_idx, l_idx)):
                lab_id = df_labs.iloc[l_idx]['Lab ID']
                current_compatible_labs.append(lab_id)

        compatible_labs.append(current_compatible_labs)

    return compatible_labs
```

6.3 Task Division

Jeryn Naidoo - 223009245

- Assisted with initial code and data setup
- Contributed towards the slides for our presentation

Kirone Gopaul - 223014456

- Initial data loading and constraint checks
- Initial development of the mathematical model of the problem

Shaista Naicker - 222106172

- Contributed towards the proposal write-up
- Contributed to the development of the presentation slides

Josh Goodwin - 222092144

- Assisted with initial code and data setup
- Assisted in the development of the mathematical model of the problem

Tashmika Pillay – 223097300

- Contributed towards the proposal write-up
- Assisted with initial code and data setup

7. INITIAL CODE PROGRESS

Data Loading

The lab and module data are defined directly as Python dictionaries and loaded into Pandas Data Frames (df_labs and df_modules). Each dictionary uses column names as keys and lists as values, which are converted into a structured table automatically.

df_labs stores each lab's ID, name, seating capacity, and a list of installed software. df_modules stores each module's code, student enrolment, and required software. Using DataFrames simplifies later constraint checking and allows the data to be easily inspected and printed for verification.

```python
labs_data = {
    "Lab ID": ["L1", "L2", "L3", "L4"],
    "Name": ["Advanced Lab A", "Coding Lab B", "Data Lab C", "Legacy Lab D"],
    "Capacity": [100, 60, 40, 30],
    "Software": [["Python", "Java", "MATLAB"], ["Python", "Java"], ["Python", "MATLAB"], ["Java", "MATLAB"]]
}

modules_data = {
    "Module Code": ["M1", "M2", "M3", "M4", "M5", "M6", "M7", "M8", "M9", "M10"],
    "Students": [85, 45, 25, 55, 35, 15, 90, 30, 20, 50],
    "Software Needed": ["Python", "MATLAB", "Java", "Python", "MATLAB", "Java", "Java", "Python", "MATLAB", "Python"]
}

df_labs = pd.DataFrame(labs_data)
df_modules = pd.DataFrame(modules_data)
```

	Lab ID	Name	Capacity	Software
0	L1	Advanced Lab A	100	[Python, Java, MATLAB]
1	L2	Coding Lab B	60	[Python, Java]
2	L3	Data Lab C	40	[Python, MATLAB]
3	L4	Legacy Lab D	30	[Java, MATLAB]

	Module Code	Students	Software Needed
0	M1	85	Python
1	M2	45	MATLAB
2	M3	25	Java
3	M4	55	Python
4	M5	35	MATLAB
5	M6	15	Java
6	M7	90	Java
7	M8	30	Python
8	M9	20	MATLAB
9	M10	50	Python

Feasibility Checks

```python
def check_fit(module_index, lab_index):

    module = df_modules.iloc[module_index]
    lab = df_labs.iloc[lab_index]

    has_space = module['Students'] <= lab['Capacity']
    has_software = module['Software Needed'] in lab['Software']

    return has_software and has_space
```

The check_fit(module_index, lab_index) function handles constraint validation. It accepts positional indices for a module and a lab, retrieves the corresponding rows from df_modules and df_labs, and then evaluates two conditions:

has_space – checks that the module's student count does not exceed the lab's capacity, enforcing the capacity constraint

has_software – checks that the module's required software appears in the lab's software list, enforcing the software compatibility constraint

The function only returns True when both conditions are satisfied simultaneously. This ensures that no infeasible module–lab assignment is considered valid.

Early Assignment Logic

No assignment logic has been implemented yet in this phase. The current code focuses entirely on data representation, constraint enforcement, and feasibility reporting. The Compatible Labs column provides the information the greedy heuristic will need to begin making assignments in Phase 2. All decisions regarding module assignment and unused seat minimisation are intentionally deferred to later phases once the model and constraints have been fully validated.

```
def display_compatability_info():
    compatible_labs = []

    for m_idx in range(len(df_modules)):

        current_compatible_labs = []

        for l_idx in range(len(df_labs)):

            if(check_fit(m_idx, l_idx)):
                lab_id = df_labs.iloc[l_idx]['Lab ID']
                current_compatible_labs.append(lab_id)

        compatible_labs.append(current_compatible_labs)

    return compatible_labs
```

	Module Code	Students	Software Needed	Compatible Labs
0	M1	85	Python	[L1]
1	M2	45	MATLAB	[L1]
2	M3	25	Java	[L1, L2, L4]
3	M4	55	Python	[L1, L2]
4	M5	35	MATLAB	[L1, L3]
5	M6	15	Java	[L1, L2, L4]
6	M7	90	Java	[L1]
7	M8	30	Python	[L1, L2, L3]
8	M9	20	MATLAB	[L1, L3, L4]
9	M10	50	Python	[L1, L2]

8. RISKS AND MITIGATION

Risk 1 – Greedy Producing High Seat Wastage

The greedy approach makes the best decision it can at each step but never goes back to reconsider earlier choices. This is a known limitation of greedy algorithms generally. There's no guarantee the result is globally optimal. For this specific dataset it's less of a concern since each module's valid lab options are quite limited once both constraints are applied, so there aren't many wrong choices to make anyway. But it's worth acknowledging as a general risk, and it's the motivation behind including a local search phase later to verify and potentially improve on whatever the greedy produces.

Risk 2 – Limited Lab Options Leading to Infeasible Assignments

Some modules have very few valid labs. M7 for example (90 students, Java) only really fits in L1. If the algorithm isn't careful about the order in which it checks labs, it could attempt incompatible ones first before landing on the right one, which could cause errors. We deal with this by running a compatibility check before any assignments start, so problems get caught early and clear errors are raised listing exactly which constraints are conflicting.

Risk 3 – Tie-Breaking on Extended Datasets

If this tool were used on a different dataset where two labs had the same capacity, the greedy would have no natural way to pick between them and the wrong choice could affect later assignments. We'd handle that by defining a clear tie-breaking rule in the code — for example, ordering alphabetically by lab ID — to keep results consistent and reproducible.

9. DECLARATION

We confirm that:

- This work is our own.
- All group members contributed as stated.

Signed:

- Jeryn Naidoo
- Shaista Naicker
- Kirone Gopaul
- Josh Goodwin
- Tashmika Pillay

Date: 24/02/2026